

LANGUAGE TUTOR

Project Proposal & Technical Development Plan

React Native + Expo | FastAPI | PostgreSQL / Supabase | AI

Version 1.0 — September 2026

A structured development plan for building a modern, AI-assisted mobile language-learning application.

1. Project Overview

Language Tutor is a mobile language-learning application designed to help users learn a new language through structured lessons, vocabulary, grammar, pronunciation, speaking practice, progress tracking, and an AI-powered tutor. The application will use React Native with Expo for the mobile frontend and FastAPI/Python for the backend.

The uploaded reference UIs establish the initial visual direction: a clean, modern, friendly learning experience with a tutor/avatar, learning-path lessons, practice categories, progress features, and a profile area.

2. Project Vision

The goal is to create a practical language tutor that does more than display static lessons. The system should adapt learning content to the user's level, interests, progress, and areas that need improvement.

- Provide structured language lessons.
- Support grammar, vocabulary, pronunciation, and speaking practice.
- Provide an AI tutor for interactive conversations.
- Give immediate educational feedback and corrections.
- Track learning progress, scores, XP, and streaks.
- Personalize recommended practice activities.
- Allow the architecture to grow toward voice-based and AI-assisted learning.

3. UI/UX Direction

The four uploaded reference screenshots are used as inspiration rather than as a direct copy. The application should preserve useful interaction patterns while developing its own visual identity.

Screen	Reference Direction	Planned Function
Home	Tutor banner, current lesson, Start action	Daily learning entry point and recommendations
Lessons	Visual learning path with lesson nodes	Structured course progression
Profile	Tutor, language, level, interests, progress	Account and learning preferences
Practice	Grammar, pronunciation, vocabulary and categories	Targeted practice activities

4. Main Application Modules

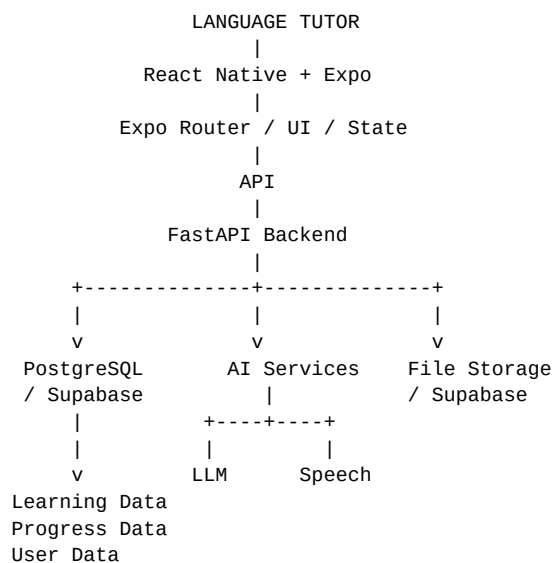
- Home / Dashboard
- Lessons and learning path
- Lesson details and exercises
- Practice hub
- Grammar practice
- Vocabulary practice
- Pronunciation practice
- Speaking practice
- AI Tutor / conversation
- User profile and preferences

- Progress and statistics
- Streaks, XP, and achievements
- Optional PRO/subscription features in a later phase

5. Recommended Technology Stack

Layer	Technology	Purpose
Mobile frontend	React Native + Expo	Android/iOS application
Language	TypeScript	Type safety and maintainable frontend code
Navigation	Expo Router	File-based application navigation
UI	NativeWind + custom components	Reusable modern UI
Local/global state	Zustand	UI and lightweight application state
Server state	TanStack Query	API fetching, caching, loading and mutation state
Backend	FastAPI + Python	REST API and AI-related services
Database	PostgreSQL	Users, lessons, progress and learning data
Backend platform	Supabase	Managed PostgreSQL, Auth and Storage
Authentication	Supabase Auth	Registration, login and session management
File storage	Supabase Storage	Images, audio and user-uploaded files
AI	LLM API	Tutor conversations and educational feedback
Speech	Speech-to-text + text-to-speech	Voice learning
Caching	Redis — later	Optional high-speed caching/session workloads
Deployment	Docker + cloud hosting	Repeatable backend deployment
Version control	Git + GitHub	Source control and collaboration

6. Proposed System Architecture



The initial implementation should remain simple. A modular FastAPI backend is preferable to starting with microservices. AI-specific services can be separated later if scale or maintenance requirements justify it.

7. Mobile Application Structure

```

mobile/
  app/
    _layout.tsx
    index.tsx
    (auth)/
      login.tsx
      register.tsx
      onboarding.tsx
    (tabs)/
      _layout.tsx
      home.tsx
      lessons.tsx
      practice.tsx
      profile.tsx
    lesson/
      [id].tsx
    practice/
      grammar.tsx
      vocabulary.tsx
      pronunciation.tsx
      speaking.tsx
    tutor/
      conversation.tsx
  components/
  services/
  stores/
  types/
  hooks/
  constants/
  assets/

```

8. Backend Structure

```

backend/
  app/
    api/
    core/
    models/
    schemas/
    services/
    main.py
  tests/
  requirements.txt
  .env
  README.md

```

The backend should separate API routes, database models, request/response schemas, business logic, authentication/configuration, and AI-related services.

9. Initial Database Design

The database should be introduced gradually. The first implementation can begin with:

- profiles
- languages
- lessons
- lesson_sections
- user_progress

The extended model can later include:

- user_languages
- levels

- tutors
- lesson_questions
- lesson_answers
- vocabulary
- user_vocabulary
- practice_categories
- practice_exercises
- lesson_progress
- streaks
- achievements
- conversations
- conversation_messages
- subscriptions

10. Example Core Tables

profiles

id, user_id, display_name, native_language, target_language, level, tutor_id, created_at

lessons

id, title, description, level, category, order_index, image_url, estimated_minutes

lesson_sections

id, lesson_id, type, question, content, answer, order_index

user_progress

id, user_id, lesson_id, progress, completed, score, last_accessed

11. Authentication Plan

- Start with email/password authentication.
- Persist authenticated sessions securely.
- Connect authenticated users to profile and progress records.
- Add Google/Apple login later if required.
- Use database access policies so users can access only permitted records.

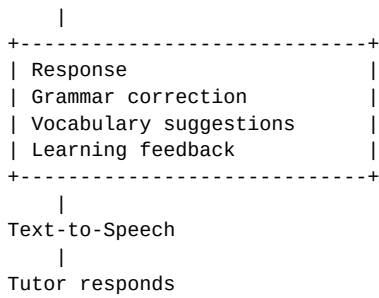
12. AI Tutor Concept

The AI Tutor should be designed as an educational tutor rather than a generic chatbot.

```

User speaks
|
Speech-to-Text
|
FastAPI
|
AI Tutor

```

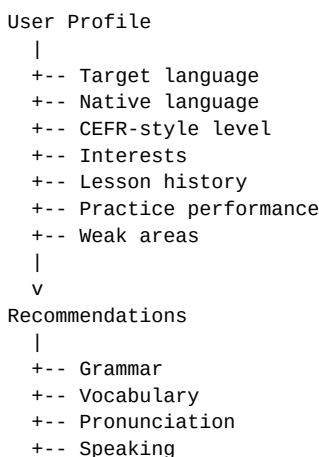


Example learning interaction:

- Tutor asks the learner a question.
- Learner responds by text or voice.
- The system detects useful corrections.
- The tutor explains the correction at the learner's level.
- The learner retries the sentence.
- The interaction contributes to progress data.

13. Personalization

The application should use learning data to recommend practice without making the interface feel complex.



14. Gamification

- XP for completed learning activities.
- Daily/weekly streaks.
- Achievement badges.
- Lesson completion indicators.
- Progress percentages.
- Daily learning goals.
- Optional leaderboard or social features later.

15. Development Roadmap

Phase 1 — Foundation

- 1 Set up React Native + Expo + TypeScript.
- 2 Configure Expo Router.
- 3 Create project folders.
- 4 Set up theme and reusable UI components.
- 5 Create bottom navigation.

Phase 2 — UI/UX

- 1 Home screen.
- 2 Lessons screen.
- 3 Practice screen.
- 4 Profile screen.
- 5 Lesson details screen.
- 6 Practice details screen.

Phase 3 — Backend & Database

- 1 Create Supabase project.
- 2 Create PostgreSQL schema.
- 3 Set up FastAPI.
- 4 Create database connection.
- 5 Create initial API endpoints.

Phase 4 — Authentication

- 1 Registration.
- 2 Login.
- 3 Logout.
- 4 Session persistence.
- 5 Profile creation.

Phase 5 — Learning Engine

- 1 Lesson content.
- 2 Questions and answers.
- 3 Scoring.
- 4 Lesson completion.
- 5 Progress tracking.

Phase 6 — Practice

- 1 Vocabulary.
- 2 Grammar.

- 3 Pronunciation.
- 4 Speaking.
- 5 Topic/category practice.

Phase 7 — AI Tutor

- 1 Text conversation.
- 2 AI-generated responses.
- 3 Grammar correction.
- 4 Personalized feedback.
- 5 Speech-to-text.
- 6 Text-to-speech.

Phase 8 — Advanced Features

- 1 Recommendations.
- 2 Achievements.
- 3 Streaks.
- 4 Notifications.
- 5 Offline learning.
- 6 PRO/subscription features.
- 7 Analytics.
- 8 Production optimization.

16. Development Strategy

The project will be developed incrementally. UI, backend, and AI should not all be implemented simultaneously at the beginning. Each milestone should produce a working result.

- Build and test one feature at a time.
- Keep reusable UI components separate from screen-specific code.
- Use TypeScript types for API and domain models.
- Keep API calls inside service modules rather than directly inside every screen.
- Use environment variables for secrets and API configuration.
- Commit working milestones to Git frequently.
- Update documentation when architecture or API behavior changes.
- Add automated tests progressively, beginning with important backend logic.

17. Git & Documentation Practice

Maintaining documentation alongside development is recommended. It makes the project easier to understand, debug, present, maintain, and continue after a break.

docs/

```
■■■■ PROJECT_OVERVIEW.md
■■■■ REQUIREMENTS.md
■■■■ ARCHITECTURE.md
■■■■ DATABASE.md
■■■■ API.md
■■■■ UI_UX.md
■■■■ DEVELOPMENT_ROADMAP.md
■■■■ AI_TUTOR.md
■■■■ TESTING.md
■■■■ DEPLOYMENT.md
```

Recommended Git workflow:

- Create a feature branch for meaningful features.
- Use clear commit messages such as: feat: add lesson screen.
- Keep commits small enough to understand.
- Update relevant documentation when architecture or API behavior changes.
- Merge tested features into the main branch.

18. Initial Milestone

MILESTONE 01

Language Tutor

■

■■■■ Home

■■■■ Lessons

■■■■ Practice

■■■■ Profile

Focus:

React Native + Expo

TypeScript

Expo Router

Reusable components

Navigation

Initial visual design

Not yet:

Database

Authentication

AI

Payments

Complex backend

19. Immediate Next Steps

- 1 Open the existing Git repository.
- 2 Confirm the current repository structure and whether a React Native/Expo project already exists.
- 3 Create or configure the mobile application using Expo + TypeScript.
- 4 Install the core frontend dependencies.
- 5 Create the app folder structure.
- 6 Create the tab navigation: Home, Lessons, Practice, Profile.
- 7 Create the shared theme and reusable components.
- 8 Implement the Home screen based on the uploaded reference.
- 9 Run the application and verify it on a physical device or emulator.
- 10 Commit the first working milestone to Git.

20. Project Principles

- Build simple features first and expand gradually.
- Keep the code modular and reusable.
- Treat the database as a source of structured learning content, not hardcoded screen data.
- Keep AI functionality behind backend APIs so provider-specific details are not exposed in the mobile app.
- Design for accessibility and clear learning feedback.
- Protect authentication credentials, API keys, and service secrets.
- Document important decisions as the project develops.
- Test each milestone before moving to the next one.

21. Conclusion

The proposed Language Tutor architecture provides a practical path from an initial React Native prototype to a full AI-assisted language learning platform. React Native + Expo will provide the mobile experience, FastAPI will provide a flexible Python backend, and PostgreSQL/Supabase will provide the core data, authentication, and storage services. AI and voice capabilities can then be added incrementally without redesigning the entire application.

The project should proceed milestone by milestone, with documentation updated alongside implementation. This approach provides a clear record of requirements, architecture, database decisions, APIs, testing, and future improvements.